

FREE COMMUNITY RESOURCE

The Placement Formula.

The 4-step system that gets freshers placed: exact actions, clear finish lines, a 90-day plan, the referral template and where the openings actually are.

A resource by @codewithswaroop x @niihaarr

The 4-step formula

Four steps, done in this exact order. Each step feeds the next: the language feeds DSA, DSA plus core feeds interviews, the project feeds the resume, and everything feeds the application machine in Step 4. Students fail not because they skip work, but because they do the right work in the wrong order, projects before basics, AI tools before fundamentals, applications before proof.

Step	When	Output at the end
1. One language + DSA	Weeks 1 to 6, then daily forever	Patterns solved, 2 problems a day habit
2. Core subjects + aptitude	Weeks 4 to 12, overlaps Step 1	80%+ on timed mocks, core answers in your own words
3. Deployed project + ATS resume	Weeks 8 to 16	Live project link + one-page resume with numbers
4. Smart apply	Week 12 until offer	10 applications, 5 referrals, 1 mock, every week

The order is the formula. Doing Step 3 without Step 1 gives you a project you cannot defend. Doing Step 4 without Step 3 gives you applications with nothing to show. Follow the sequence.

SECTION 3

Step 1: One language + DSA

The classic mistake: switching languages every month and solving random problems. The fix is one language and pattern-wise practice. **150 pattern-wise problems teach more than 300 random ones.**

What exactly to do

- **Pick ONE language and stop debating.** Python if you want speed and AI or data options later. Java if you target service companies and enterprise roles. Both are accepted in almost every interview.

- **3 weeks on syntax, no more.** Variables, loops, functions, OOP basics, collections. Write code daily. Watching videos alone teaches you nothing you can reproduce.
- **Then patterns, one at a time.** Learn one pattern, solve 8 to 10 problems of it, move on.
- **Daily minimum: 2 problems.** One new, one revision. The streak matters more than the volume.

The patterns that cover most interviews

Pattern	What it solves
Arrays + hashing	Frequency counts, lookups, duplicates
Two pointers	Sorted array pairs, palindromes, partitioning
Sliding window	Longest or shortest subarray with a condition
Binary search	Sorted data, and answers you can guess-and-check
Stack + queue	Matching brackets, next greater element, scheduling
BFS / DFS	Trees, graphs, islands, connected components
Recursion + backtracking	Combinations, permutations, sudoku-style search
Heaps	Top K problems, streaming medians
Dynamic programming basics	Choices with overlapping subproblems

Resources

- **freeCodeCamp** (freecodecamp.org): hands-on curriculum with free certifications, use the Python or JavaScript track for the syntax phase.
- **NeetCode Roadmap** (neetcode.io/roadmap): 150 problems arranged as a pattern tree, follow the tree order, free video solution for every problem.
- **Striver A2Z DSA Sheet** (takeuforward.org): the most-followed DSA sheet for Indian placements, basics to advanced.

STEP 1 IS DONE WHEN you solve a new easy problem in under 15 minutes and a familiar-pattern medium in under 40, without looking at solutions.

SECTION 4

Step 2: Core subjects + aptitude

Two different filters live in this step. Aptitude tests eliminate most candidates before any human reads a resume, and core CS questions decide the technical interview after your coding round. Prepare both in parallel with Step 1.

The questions that actually get asked

Subject	Be ready to answer, out loud, in your own words
DBMS	Why indexes make queries fast. Normalization. ACID. INNER vs LEFT JOIN. Why SELECT * is discouraged. SQL vs NoSQL trade-offs.
Operating Systems	Process vs thread. Deadlock and its four conditions. Virtual memory and paging. Context switch. Mutex vs semaphore.

Networks	What happens when you type google.com and press enter. TCP vs UDP. HTTP vs HTTPS. DNS. The three-way handshake.
OOP	The four pillars, each with an example from your own project, never a textbook definition.
Aptitude	Percentages, profit and loss, time-speed-distance, ratios, puzzles. 10 minutes daily on timed sets.

Resources

- **SWAYAM / NPTEL** (swayam.gov.in): real IIT courses on DBMS, OS and Networks, free to learn. An NPTEL Elite or Gold rank is a genuinely strong fresher resume line.
- **CS Roadmap** (roadmap.sh/computer-science): a visual map of every core area, use it to find blind spots.
- **IndiaBix** (indiabix.com): free timed aptitude tests with worked explanations for every answer.
- **Aptitude + DBMS Course, Telugu** (codewithswaroop.com): both first-round filters in one structured Telugu course, for those who want it guided.

STEP 2 IS DONE WHEN you score 80%+ on a timed aptitude mock and can explain indexing, deadlock, and TCP vs UDP out loud in under a minute each.

SECTION 5

Step 3: Deployed project + ATS resume

Recruiters see thousands of resumes with a to-do app, a calculator and a portfolio site. One project that is live on the internet, uses a real database, and has one genuinely hard feature beats ten tutorial clones.

The 3 conditions for a project that counts

- **It is deployed.** A URL that opens on a recruiter's phone. Free hosting exists: Vercel, Render, Railway. Localhost is a demo, not a project.
- **It uses real data.** A real database with create, read, update, delete. Not a hardcoded array.
- **One feature was hard.** Authentication with roles, payments, search, file upload, or a real-time update. You should be able to talk about it for 10 minutes, because interviewers will.

Resume lines: weak vs strong

Weak line	Strong line
Worked on an e-commerce project	Built an e-commerce platform with 200+ products, payments and search, deployed on AWS, 300+ users
Knows React, Node, MongoDB	Shipped a MERN notes app used by 200 students, live link and GitHub on request
Improved application performance	Cut page load from 4.1s to 1.3s by adding caching and image compression

The formula for every bullet: Built X using Y, achieving Z. Single column, one page, no photo, no icons, no tables. GitHub link and live link mandatory. Free ATS builder: rxresu.me. Free real projects to build: github.com/practical-tutorials/project-based-learning (274K stars).

STEP 3 IS DONE WHEN your project link opens on mobile, the README explains the hard feature, and every resume line is either a fact with a number or a skill you can be grilled on.

SECTION 6

Step 4: Smart apply

Applying is a system, not an event. The same resume gets dramatically different results depending on the channel. In our own 200-application experiment, the numbers looked like this:

Channel	Applications	Callbacks	Rate
LinkedIn Easy Apply	100	3	3%
Direct on company website	50	11	22%
Referral through a connection	50	24	48%

Same person, same resume, 16x difference. That is why the weekly system below spends effort where it converts.

The weekly system, every week until offer

- **10 quality applications** through official careers pages. Find the role anywhere, apply on the company's own site.
- **5 referral requests.** Find an employee of the target company on LinkedIn, alumni first, and send the template below.
- **1 mock interview** on Pramp (pramp.com, free) or with a friend. Thinking out loud is a separate skill from solving.
- **Track everything in one sheet:** company, role, link, date, channel, status, follow-up date. Follow up once after 7 to 10 days.

The referral message, copy and edit

Hi [Name], I'm a final-year [branch] student applying for the [Role] opening at [Company] (Job ID: [ID]). I've built [one-line project with link] and I've prepared seriously for this role. If you feel my profile fits, could you refer me? Happy to send my resume. Either way, thank you for your time.

STEP 4 IS DONE WHEN you have an offer. Until then the weekly system runs every single week. No skipped weeks.

SECTION 7

Mistakes that cost people the offer

Mistake	What it looks like	Fix
Waiting for campus placements	"My placement cell will handle it"	Off-campus applications start 6 months before season
Tutorial hell	20 courses watched, nothing built	Pause the video, rebuild the code without it
Ten shallow projects	To-do app, calculator, portfolio, clone, clone	One deployed project with one hard feature
Language hopping	Python this month, Java next month	One language, 3 weeks syntax, then patterns

Random problem grinding	300 problems, no pattern awareness	Pattern-wise sheets, 8 to 10 problems per pattern
Decorated resume	Two columns, photo, icons, skill bars	Single column, one page, numbers in every bullet
Easy Apply only	100 one-click applications, silence	Careers pages plus referrals, 10 + 5 weekly
Silent interviews	Solving in your head, then saying the answer	Narrate every step, weekly mocks until it is natural

SECTION 8

The 90-day version

Placement season close? The same formula, compressed. It assumes 2 to 3 focused hours a day.

Days	Focus	Target by the end
1 to 21	Language syntax + first DSA patterns. Aptitude 10 min daily from day 1.	60+ problems, arrays and two-pointers comfortable
22 to 45	DSA patterns + DBMS and OS. Project starts on weekends.	120+ problems, SQL confident, project skeleton deployed
46 to 70	Finish the hard feature. Networks + OOP. Resume built.	Live project + ATS resume, 170+ problems
71 to 90	Full Step 4 system: applications, referrals, mocks, timed tests.	40+ quality applications out, interview-ready

Where the openings are

Snapshot verified 26 Aug 2026, always re-check dates on the official page. Deloitte: apply.deloitte.com (analyst roles, 700+ fresher listings on Naukri that month). Accenture: accenture.com/in-en/careers (Associate Software Engineer, off-campus). KPMG: kpmg.com/in/en/careers.html (three application portals). TCS: tcs.com/careers/india and the iBegin portal (NQT route, 3.6 to 9 LPA by tier). Amazon: amazon.jobs (SDE-1 university hiring). Check weekly: careers.microsoft.com, google.com/about/careers, careers.wipro.com.

The whole guide in three lines. One language, then patterns. Core plus aptitude while you build one real project. Then 10 applications, 5 referrals, 1 mock, every week until the offer.

The college does not decide the package. The floor you sit on does. Idi luck kaadu. Idi formula.

If this helped you, send it to one friend who has placements coming up. That is the only thing we ask.
@codewithswaroop x @niihaaarrrr